DNSキャッシュポイズニング攻撃抑制を目的としたTCP Fallback環境の構築と性能評価

田井 豊浩[1,a)]　喜村 柊太[1,b)]　小林 孝史[2,c)]

概要：現在，DNS（Domain Name System）はインターネットの重要な基盤技術の 1 つとして広く利用されている．しかし，DNS の重要性が増すとともに DNS サーバへの攻撃が増加しており，DNS キャッシュポイズニング攻撃をはじめとする脅威への対策が求められている．本研究では，DNS キャッシュポイズニング攻撃の抑制を目的として，応答パケットに TC（Truncated）フラグを付与し，TCP 接続へ誘導する DNS サーバを実装した．さらに，実装した DNS サーバのスループット等を計測し，通常の DNS サーバとの比較により TCP Fallback 環境における性能への影響を評価した．また，TCP Fallback による性能の変化を明らかにするとともに，実運用に向けた適用可能性について考察した．

キーワード：DNS キャッシュポイズニング，TCP Fallback，TC フラグ，スループット

Implementation and Performance Evaluation of a TCP Fallback Environment for Mitigating DNS Cache Poisoning Attacks

Toyohiro Tai[1,a)]　Shuta Kimura[1,b)]　Takashi Kobayashi[2,c)]

Abstract: The Domain Name System (DNS) is one of the fundamental technologies supporting the Internet and is widely used worldwide. However, as the importance of DNS continues to grow, attacks targeting DNS servers have become increasingly prevalent, making countermeasures against threats such as DNS cache poisoning attacks essential. In this study, we implemented a DNS server that sets the Truncated (TC) flag in DNS response packets to induce TCP connections to mitigate DNS cache poisoning attacks. Furthermore, we measured the throughput of the implemented DNS server and evaluated the impact of the TCP fallback environment on performance by comparing it with a conventional DNS server. In addition, we clarified the performance changes caused by TCP fallback and discussed the feasibility of applying the proposed approach in practical environments.

Keywords: DNS Cache Poisoning,TCP Fallback,TC Flag,Throughput

1. はじめに

現在，DNS（Domain Name System）は，インターネットの重要な基盤技術の１つとして広く利用されている．DNSとは，ネットワーク上の通信で必要な数字である IP アドレスを人が扱いやすいドメイン名と紐づける技術として，現在のネットワーク利用において無くてはならない存在となっている．DNS ではドメインの委任という仕組みでドメイン名と IP アドレスのマッピングを階層構造に分散化させており，ドメインの追加などの更新作業を容易にすると共に負荷分散の効果を持つ．こういった工夫により DNS は急速に成長するインターネットに対応し続けている．

しかし，DNS の重要性が増すにつれて DNS が悪意のある攻撃の対象にされるケースが増えている．DNS を狙っ

[1] 関西大学大学院 総合情報学研究科
Intelligent Informatics Major, Graduate School of Informatics, Kansai University Graduate School
[2] 関西大学 総合情報学部
Faculty of Informatics, Kansai University
[a)] k745666@kansai-u.ac.jp
[b)] k149017@kansai-u.ac.jp
[c)] taka-k@kansai-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

た攻撃は古くからあり，特に DNS キャッシュポイズニング攻撃の危険性は長年指摘されている．これは，DNS の名前解決を高速化するために用いるキャッシュ情報に不正なドメイン名と IP アドレスを紐づけさせる攻撃手法である．DNS キャッシュポイズニングが成功するとキャッシュの利用者を悪意あるサービスに誘導することが可能となる．この攻撃によって，フィッシングサイトを始めとする有害サイトなどの偽装応答先に誘導されると詐欺やマルウェア感染などの実害が発生する可能性がある．

この攻撃の対策手法として DNS におけるセキュリティ拡張機能である DNSSEC（DNS Security Extensions）の導入が挙げられる．DNSSEC とは電子署名を用いてリソースレコードが改竄されていないことを検証する仕組みであり，DNS キャッシュポイズニング攻撃の根本的な解決策として注目された．しかし，後述のように実装や管理の複雑さ，キャッシュサーバに対する負荷増加など課題が多く存在する．そのため，DNSSEC とは異なるアプローチによる対策について検討が行われている．

Afek らは，DNS キャッシュポイズニング攻撃に対し，TCP による名前解決を用いることで攻撃成功率を大幅に低減できることを示した [1]．TCP による名前解決ではコネクション確立が必要となるため，UDP を標的としたオフパス攻撃による偽装応答の挿入が極めて困難となることを理由としている．しかし，TCP による名前解決は UDP による名前解決と比較して通信遅延やサーバ負荷の増加を招くという課題がある．したがって，TCP による攻撃抑制効果を DNS 環境へ適用する場合，TCP を利用することによる性能への影響を考慮する必要がある．

本研究では，先行研究で示された DNS キャッシュポイズニング攻撃に対する TCP による抑制手法を実際の DNS 環境へ適用することを想定し，TCP Fallback を利用した名前解決手法を検討する．そのために，DNS 応答パケットに TC（Truncated）フラグを任意に付与する機能を実装し，キャッシュ DNS サーバが必要に応じて TCP Fallback を能動的に誘導できる方式を構築する．これにより，通常時は UDP による高速な名前解決を維持しながら，必要な場合のみ TCP による名前解決へ移行することを可能とする．

さらに，TC フラグ付与機能を DNS サーバの内部及び外部に実装し，それぞれの実装方式における名前解決性能を比較・評価する．これにより，先行研究で示された TCP による DNS キャッシュポイズニング攻撃の抑制手法を実際の DNS 環境へ適用する際の性能への影響を明らかにし，実装方式の違いによる性能および適用可能性について検討する．

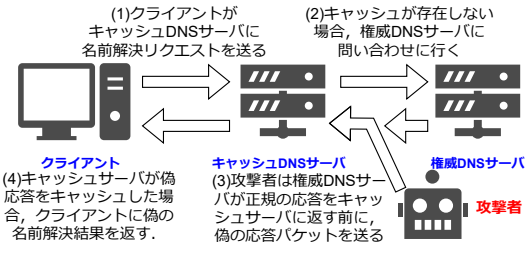


図 1　DNS キャッシュポイズニング攻撃の流れ

2. DNS の仕組みと関連技術

2.1 DNS の仕組み

DNS（Domain Name System）は OSI 参照モデルにおけるアプリケーション層に存在する分散型のデータベースであり，ドメイン名と IP アドレスを紐づけて管理するシステムとして広く使われている．ドメイン名を IP アドレスに変換する行為を名前解決と呼び，通信効率の観点から基本的には UDP ポートを使用している．また，DNS における Well Known Ports は 53 番を使用している [2]．

なお，一般的に DNS サーバの構築には BIND（Berkeley Internet Name Domain）が使われることが多い．これは権威サーバとキャッシュサーバ（リゾルバ）の役割を一つのプログラム（named）として，兼用しており，機能が豊富であるという特徴があるからである．

2.2 DNS キャッシュポイズニング攻撃

DNS では，キャッシュという仕組みを利用して，権威 DNS サーバの負荷の軽減や応答の高速化を図っている．DNS キャッシュポイズニングとは，悪意のある攻撃者がキャッシュ DNS サーバに偽の DNS 応答を送り込み，その偽の DNS 情報をキャッシュ DNS サーバにキャッシュとして保持させ，クライアントに偽の DNS 情報を返させる攻撃手法である．図 1 のように DNS キャッシュポイズニング攻撃を成功させるには，攻撃者はキャッシュ DNS サーバに対して，偽造した応答を正当な応答よりも早く送り込む必要がある．このとき，応答を偽造するには，送信元，宛先 IP アドレス，宛先ポート番号，問い合わせポート番号，問い合わせドメイン・クラス・タイプ，トランザクション ID を正当な応答と一致させる必要がある．

DNS キャッシュポイズニングの成功率は式 (1) で表される [3]．

$$P = \frac{R \times W}{N \times \mathrm{Port} \times \mathrm{ID}} \tag{1}$$

ここで，R は攻撃者が 1 台のキャッシュサーバに送信するパケットのレート（パケット毎秒, pps），W は攻撃可能な時間ウィンドウ（クエリから応答までの往復遅延時間, RTT），N は攻撃対象のレコードを保持する権威 DNS サーバの数，Port はキャッシュサーバが使用するクエリ送信元ポートの数，ID は DNS トランザクション ID の総数（通常 16 ビットで 65,536 通り）である．R や W が大きいほど攻撃の成功率は上がり，N や ID が多いほど成功率は下がる．

© 2026 Information Processing Society of Japan

2.3 DNSSEC（DNS Security Extensions）

DNSSEC は，公開鍵暗号方式の電子署名技術を基に DNSSEC キャッシュサーバによる問い合わせへの応答が正当であるかを検証するために利用する．応答に対してゾーンの管理者が登録したレコードを下に認証を行い，応答内容が正当である場合に名前解決を完了させる．DNS キャッシュポイズニングは，正当ではない DNS サーバから偽装された応答パケットを送る攻撃であるため，DNSSEC を用いると，DNS キャッシュポイズニングを防ぐ事ができる．

DNSSEC では，ハッシュ関数を用いてハッシュ値を計算し，得られたハッシュ値を秘密鍵で暗号化を行い，電子署名が格納されるリソースレコードである RRSIG（Resource Record digital SIGnature）を生成する．権威 DNS サーバは，キャッシュ DNS サーバからの問い合わせを受け取ると，リソースレコードと，その RRSIG を応答としてキャッシュ DNS サーバに返す．権威 DNS サーバからの応答を受け取ったキャッシュ DNS サーバはハッシュ関数を用いてリソースレコードからハッシュ値を計算する．また，RRSIG をリソースレコードを署名する秘密鍵に対応する公開鍵である DNSKEY を用いて復号してハッシュ値を求める．リソースレコードから計算したハッシュ値と RRSIG を復号して得たハッシュ値が一致した場合，DNSSEC 検証が成功して，権威 DNS サーバからの応答が正当であると分かる [4]．

2.4 DNSSEC の問題点

阿久津らは，DNSSEC の課題として，DNS 応答サイズの増大に伴う通信上の問題や，フラグメンテーションに起因するセキュリティ上の問題などを指摘している [5]．本節では，これらの課題について整理する．

(1) パケットサイズの増加

従来の DNS では，UDP を用いて送信可能な DNS メッセージのサイズは 512Byte に制限されていた．しかし，DNSSEC では DNSKEY や RRSIG などのレコードが応答に追加されるため通常の DNS と比較して応答サイズが大きくなる．その結果，DNSSEC を利用した応答が 512Byte を超える場合がある．このような大きな DNS 応答を UDP で扱うために EDNS0（Extension Mechanisms for DNS）が利用される．EDNS0 ではクライアントが受信可能な UDP ペイロードサイズを通知することで，512Byte を超える DNS 応答を UDP で送信することが可能となる [6]．一方，EDNS0 に対応していない場合や，ネットワーク上の制約によって大きな UDP 応答を受信できない場合には，DNS 応答が切り詰められ，TCP による再問い合わせが必要となることがある．TCP 通信では接続確立などの処理が必要となるため，UDP と比較して通信処理の負荷や遅延が増加する可能性がある．

(2) 相互運用性の問題

DNSSEC によって DNS 応答が大きくなると，ネットワーク上で設定された Path MTU を超過し，IP フラグメンテーションが発生する可能性がある．また，ファイアウォールなどの中間装置がフラグメント化された UDP パケットを適切に処理できない場合，DNS 応答を正常に受信できない可能性がある．実際に，DNS 応答のサイズが大きくなることで，ネットワーク上の機器によるパケットの遮断や処理上の問題が発生することが報告されている [6]．このように，DNSSEC による応答サイズの増加は DNS サーバやクライアントだけでなく通信経路上のネットワーク機器との相互運用性にも影響を及ぼす．

(3) フラグメンテーションを利用した攻撃

DNS 応答が IP フラグメンテーションを起こした場合，DNS ヘッダなどの情報を含む最初のフラグメントとそれ以降のフラグメントに分割されて通信が行われる．この構造を悪用し，後続フラグメントを偽装することで DNS キャッシュポイズニングを試みる攻撃手法が報告されている [7][8]．したがって，DNSSEC そのものが直接的に攻撃を引き起こすというよりも，DNSSEC による応答サイズの増加によってフラグメンテーションが発生しそのフラグメント処理の特性が攻撃に利用される可能性がある．

(4) リフレクション攻撃

DNSSEC では DNS 応答に電子署名などの情報が追加されるため通常の DNS と比較して応答サイズが大きくなる．攻撃者が送信元 IP アドレスを偽装して DNS 問い合わせを送信した場合，この大きな応答を被害者に集中させることで DNS を利用したリフレクション型の DoS 攻撃に悪用される可能性がある．そのため，DNSSEC の普及に伴う DNS 応答サイズの増大は DNS を利用したリフレクション攻撃の増幅効率を高める要因の一つとなる．

このような背景から DNSSEC の普及は遅れを取っている．APNIC Labs が提供している DNSSEC 検証率の統計地図 [9] によると全世界での DNSSEC 検証率は 2026 年 8 月 12 日現在 38.37%となっており，日本における DNSSEC 検証率はで 14.69%となっている．

また，これらの課題点に加え，内閣官房国家サイバー統括室（NCO）は原則 2035 年までに耐量子暗号への移行する方針を示している [10]．現在 DNSSEC では RSA や ECDSA などの公開鍵暗号方式を利用して電子署名を実現しているため，将来的には耐量子暗号方式への移行が必要になると考えられる．しかし，現在有力視されている耐量子署名方式は従来の公開鍵暗号方式と比較して公開鍵や電子署名の

© 2026 Information Processing Society of Japan

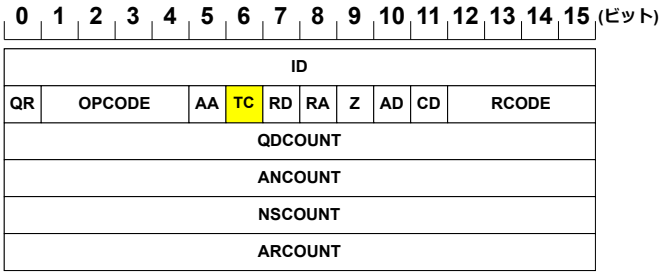


図 2　DNS ヘッダにおける TC フラグの位置

サイズが大きいものが多く，DNS 応答サイズの増加を招く可能性がある．その結果，EDNS0 や TCP Fallback への依存，フラグメント化の発生，通信遅延などの DNSSEC が抱える課題がさらに顕在化することが懸念される．

2.5　TC（Truncated）フラグ

TC フラグは，DNS ヘッダに含まれるフラグの一つであり，DNS 応答メッセージが UDP で送信可能なサイズを超えたために応答が途中で切り詰められたことを示すフラグである．権威 DNS サーバまたはキャッシュ DNS サーバは，応答を UDP で送信できない場合に TC フラグを 1 に設定して応答を返す．TC フラグが設定された応答を受信したリゾルバは完全な応答を取得するため TCP を用いて再度名前解決を行う．この一連の動作を TCP fallback と呼ぶ [11].

2.6　TCP fallback

従来の DNS では，UDP で送信できる DNS 応答サイズは 512Byte までに制限されており，この制限は「512Byte の壁」と呼ばれていた．そのため，DNS 応答が 512Byte を超える場合には DNS サーバは TC フラグを設定した応答を返し，リゾルバは TCP を用いて再度名前解決を行う必要があった．

その後，EDNS0 が標準化され，クライアントとサーバが受信可能な UDP パケットサイズを事前に通知できるようになった．これにより，512Byte を超える DNS 応答であっても UDP による通信が可能となり，通常の名前解決において TCP Fallback が発生する機会は大幅に減少した [12].

TCP Fallback は通常の UDP による名前解決とは異なる通信状態を示すため，その発生の有無を DNS 通信に関する判断結果を伝達する情報として利用できる可能性がある．また，TC フラグを付与することでキャッシュ DNS サーバに TCP への切り替えを促すことができるため，TCP Fallback を利用して特定の DNS 通信を制御することが考えられる．このような背景から，本研究では TCP Fallback を単なる通信方式の切り替えではなく，DNS 通信における状態情報の伝達手段として利用することを目指す．その実現可能性を検討するため，TC フラグの付与による DNS サーバの性能への影響を評価し，スループットや CPU 使用率，応答遅延の観点から，TCP への切り替えを誘導した場合に実用的な性能を維持できるかを明らかにする．

3.　TC フラグを付与する名前解決環境

3.1　概要

本研究では，TCP Fallback を利用した DNS 通信制御方式の評価を目的として，TC フラグを利用した TCP Fallback 誘導機能を実装した名前解決環境を構築する．

通常，DNS では UDP を用いた名前解決が行われるが TCP Fallback が発生すると TCP による名前解決へ切り替わる．本研究では，DNS 応答に TC フラグを付与することによって任意のタイミングで TCP Fallback を発生させる機能を実装し，TCP による名前解決を利用した制御方式について評価を行う．

構築した環境では，TC フラグ付与機能を DNS サーバ外部および DNS サーバ内部に実装し，それぞれの方式における名前解決性能への影響を比較する．性能評価では，resperf を用いてスループットを測定し，vmstat を用いて CPU やメモリなどのシステム負荷を計測する．また，提案方式の評価を行うため，FreeBSD による仮想マシン環境および Raspberry Pi を用いた物理環境の 2 種類の環境を構築した．仮想マシン環境では TC フラグ付与機能の実装方式による性能差を評価し，Raspberry Pi 環境では実機上での動作性能を確認する．

3.2　DNS サーバ外部への TC フラグ付与機能の実装

図 3 に DNS サーバ外部へ TC フラグ付与機能を実装した名前解決環境を示す．本環境は，クライアント，TC フラグ付与装置およびキャッシュ DNS サーバから構成される．クライアントから送信された DNS クエリは TC フラグ付与装置を経由してキャッシュ DNS サーバへ転送される．キャッシュ DNS サーバから返送された DNS 応答は TC フラグ付与装置で受信され，DNS ヘッダの TC フラグを付与した後にクライアントへ返送される．これにより，クライアントは TCP Fallback を引き起こし，TCP による名前解決へ移行する．

TC フラグ付与装置は Go 言語を用いて実装した DNS プロキシであり，クライアントとキャッシュ DNS サーバとの間で DNS パケットを中継する．DNS パケットの送受信

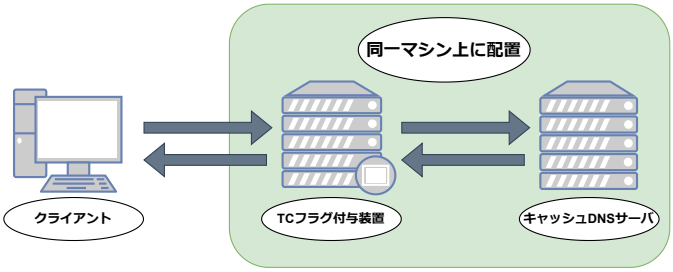


図 3　サーバ配置図

© 2026 Information Processing Society of Japan

および DNS メッセージの処理には，Go 言語の DNS ライブラリである `github.com/miekg/dns` を使用した [13]．本装置は，クライアントから受信した DNS クエリをキャッシュ DNS サーバへ転送し，応答を受信すると DNS メッセージの内容は変更せず，DNS ヘッダの TC フラグを 1 に設定してクライアントへ返送する．これにより，既存のキャッシュ DNS サーバを改造することなく任意の DNS 応答に対して TC フラグを付与し，TCP Fallback を発生させることが可能となる．本方式では，TC フラグ付与機能を DNS サーバ外部へ実装することで，既存の DNS サーバ環境へ容易に適用できるという利点を有する．

3.3 DNS サーバ内部への TC フラグ付与機能の実装

本方式では，キャッシュ DNS サーバとして BIND を利用し，BIND 内部の DNS 応答生成処理を改変することで TC フラグの付与を実現する．クライアントから送信された DNS クエリは BIND によって通常通り処理される．その後，DNS 応答の生成処理において TC フラグを設定し，クライアントへ返送することで TCP Fallback を発生させる．

BIND では，DNS メッセージのヘッダ情報は `client->message->flags` によって管理されており，TC フラグは `DNS_MESSAGEFLAG_TC` を設定することで付与される．通常，BIND では DNS 応答の生成時に応答サイズが制限値を超過した場合，`dns_message_rendersection()` が `ISC_R_NOSPACE` を返す．これを受けて `DNS_MESSAGEFLAG_TC` を設定し，UDP 応答に TC フラグを付与することで TCP Fallback を発生させる．これにより，外部装置を介さずに名前解決処理と連携した制御が可能である．一方で，DNS サーバのソースコード変更を必要とするため，既存環境へ適用する場合には BIND の改修および再ビルドが必要となる．

本研究では，この応答生成処理を改変し，`dns_message_renderbegin()` による DNS 応答生成開始後に `client->message->flags |= DNS_MESSAGEFLAG_TC` を実行することで，応答サイズに依存せず TC フラグを付与可能な機能を実装した．これにより，通常では TCP Fallback が発生しない DNS 応答に対しても，任意に TCP による名前解決へ移行させることが可能となる．

3.4 評価方法

実験に用いた環境を表 1 に示す．本研究では，キャッシュ DNS サーバ内部へ TC フラグ付与機能を実装した方式（以下，内部実装方式）と，DNS サーバ外部へ TC フラグ付与装置を配置した方式（以下，外部実装方式）について性能評価を行った．比較対象として，TC フラグ付与機能を持たないオープンソースの DNS サーバソフトウェアである BIND 9.21.23 を使用した．

表 1 評価環境

項目	内容
仮想 OS での実験環境	FreeBSD 14.2
実機での実験環境	Raspberry Pi 5
DNS サーバ	BIND v9.21.23
TC フラグ付与装置	Go 言語
スループット計測ツール	resperf
負荷監視	vmstat

評価項目は，名前解決性能およびシステム負荷とした．名前解決性能は，図 3 に示すクライアントから resperf[14] を用いて測定したスループット（QPS）および Loss 率によって評価した．QPS は，単位時間あたりに名前解決を完了した DNS クエリ数を示す．式 (2) により算出できる．

$$QPS_{\max} = \max_i \left(\frac{N_{\mathrm{complete},i}}{\Delta T_i} \right) \tag{2}$$

ここで，$N_{\mathrm{complete},i}$ は i 番目の測定区間において名前解決を完了した DNS クエリ数，ΔT_i は i 番目の測定区間の時間間隔を示す．また，Loss 率は，送信した DNS クエリのうち最終的に名前解決を完了できなかったクエリの割合を示し，式 (3) により算出した．

$$\text{Loss 率} = \frac{N_{\mathrm{sent}} - N_{\mathrm{complete}}}{N_{\mathrm{sent}}} \times 100 \tag{3}$$

ここで，N_{sent} は送信した DNS クエリ数，N_{complete} は最終的に名前解決を完了した DNS クエリ数を示す．なお，TC フラグを受信したクエリについては，TCP による再送後に応答を受信できた場合，名前解決を完了したクエリとして扱った．本来の resperf は UDP による名前解決を対象としているため，本研究では，TC フラグが設定された DNS 応答を受信した際に TCP による名前解決を追跡できるよう resperf を改造して使用した．具体的には，DNS 応答の TC ビットを検出し，UDP で送信した元のクエリを TCP で再送する処理を追加した．これにより，TCP による名前解決で応答を受信した場合についても，通常の応答と同様に名前解決を完了したクエリとして測定結果に反映した．また，システム負荷は，vmstat を用いてキャッシュ DNS サーバ上の CPU 使用率を計測した．

4. 検証結果

本検証では，内部実装方式及び外部実装方式について，仮想環境および実機環境の 2 つの環境を用いて性能評価を行った．両環境において，DNS サーバのスループットおよび CPU 使用率を測定し，TC フラグ付与による性能への影響を比較した．

4.1 仮想環境及び実機環境でのスループット計測

DNS サーバ内部および DNS サーバ外部に TC フラグ付与機能を実装した環境について，resperf を用いた性能評価

© 2026 Information Processing Society of Japan

表 2　仮想環境における性能比較

方式	QPS	Loss 率
通常の BIND	29294	0.46%
内部実装方式	14719	19.02%
外部実装方式	2268	30.71%

表 3　実機環境における性能比較

方式	QPS	Loss 率
通常の BIND	27637	6.71%
内部実装方式	17672	0.82%
外部実装方式	3565	1.69%

を行った．評価では最大送信レートを 30000 QPS に設定し，各方式について 10 回の測定を行った．表 2 に測定結果の平均値を示す．表 2 より，通常の BIND では最大スループット 29294 QPS を確認した．一方，内部実装方式では，最大スループットが 14719 QPS となり，通常の BIND と比較して約 49.75%低下した．これは，DNS 応答生成処理へ TC フラグ設定処理を追加したことによる処理負荷の増加が影響したと考えられる．また，通常の BIND では Loss 率が 0.46%であったのに対し，内部実装方式では 19.02%となり，Loss 率の増加も確認された．

また，外部実装方式では最大スループットが 2268 QPS となり，通常の BIND と比較して約 92.26%低下した．さらに，Loss 率も 30.71%となり，内部実装方式と比較しても高い値となった．これは，TC フラグ設定処理による負荷に加え，DNS サーバとクライアント間にプロキシ処理を介在させたことで，DNS パケットの受信，転送，応答処理が追加され，通信処理のオーバーヘッドが増加したためであると考えられる．

仮想環境における評価結果を踏まえ，実機環境においても各 TC フラグ付与方式の性能評価を行った．表 3 に各 TC フラグ付与方式における性能評価結果の平均値を示す．

表 3 より，実機環境においても，TC フラグ付与機能を追加することで最大スループットが低下する傾向が確認された．内部実装方式では通常の BIND と比較して約 36%低下し，外部実装方式では約 87%低下した．特に外部実装方式では，内部実装方式と比較しても大幅な性能低下が確認された．

この結果は，仮想環境において確認された TC フラグ付与方式の違いによる性能低下の傾向と一致している．また，外部実装方式において特に大きな性能低下が生じたことから，プロキシを介した TC フラグ付与処理がスループットに与える影響は，実機環境においても大きいことが確認された．

4.2　仮想環境及び実機環境における負荷計測

表 4 に仮想環境における各 TC フラグ付与方式の CPU 使用率の測定結果を示す．vmstat の結果から，user はユーザ空間で実行された処理の CPU 使用率，sys は OS カーネルによるシステム処理の CPU 使用率を示す．

表 4　仮想環境における CPU 比較

方式	user	sys	CPU 使用率
通常の BIND	9.17%	26.06%	35.23%
内部実装方式	14.23%	23.04%	37.27%
外部実装方式	31.95%	12.38%	44.34%

表 5　実機環境における CPU 使用率比較

方式	user	sys	CPU 使用率
通常の BIND	9.66%	6.56%	16.22%
内部実装方式	24.39%	16.22%	40.62%
外部実装方式	19.06%	28.91%	47.98%

仮想環境では通常の BIND の平均 CPU 使用率が 35.23%であったのに対し，内部実装方式では 37.27%，外部 TC フラグ付与方式では 44.34%となった．したがって，TC フラグ付与機能を追加した両方式において通常の BIND と比較して CPU 使用率の増加が確認された．内部実装方式では user が通常の BIND の 9.17%から 14.23%へ増加した一方，sys は 26.06%から 23.04%へ低下した．一方，外部 TC フラグ付与方式では user が 31.95%，sys が 12.38%となり，通常の BIND と比較して user の増加が確認された．

また，表 5 に実機環境における各 TC フラグ付与方式の CPU 使用率の測定結果を示す．実機環境では，通常の BIND の平均 CPU 使用率が 16.22%であったのに対し，内部実装方式では 40.62%，外部 TC フラグ付与方式では 47.98%となった．このことから，実機環境においても TC フラグ付与機能を追加した両方式で CPU 使用率の増加が確認された．内部実装方式では，user が通常の BIND の 9.66%から 24.39%へ増加した一方，sys は 6.56%から 16.22%へ増加した．外部 TC フラグ付与方式では user が 19.06%，sys が 28.91%となり，特に sys の増加が確認された．

5.　考察

5.1　TC フラグ付与による性能への影響

スループットおよび CPU 使用率の測定結果から，TC フラグ付与機能の実装方法によって性能への影響が異なることが確認された．表 6 に，通常の BIND を基準とした各方式のスループット低下率を示す．

表 6 より，内部実装方式では仮想環境において 49.75%，実機環境において 36.04%のスループット低下が確認された．一方，外部実装方式では仮想環境において 92.26%，実機環境において 87.10%の大幅なスループット低下が確認された．このことから，実行環境が異なる場合においても外部実装方式は内部実装方式と比較してスループットへの影響が大きく，TC フラグ付与方式による性能への影響は同様の傾向を示すことが分かる．

© 2026 Information Processing Society of Japan

表 6　環境ごとのスループット低下率の比較

環境	内部実装方式	外部実装方式
仮想環境	49.75%	92.26%
Raspberry Pi 5	36.04%	87.10%

以上の結果から，内部実装方式では仮想環境および実機環境の双方において user の増加が確認されており，DNS 応答生成処理に TC フラグ設定処理を追加したことによって，ユーザ空間で実行される処理量が増加したためであると考えられる．一方，外部実装方式では，仮想環境において user，実機環境において sys の増加が確認されている．これは，DNS サーバとクライアント間に配置したプロキシが DNS パケットの受信，解析および転送を行うことで，DNS サーバ内部での処理に加えてプロキシによる追加処理が発生し，CPU 負荷が増加したためであると考えられる．

また，仮想環境と実機環境では CPU 使用率の絶対値に差があるものの，TC フラグ付与機能を追加した方式では通常の BIND と比較して CPU 使用率が増加するという傾向が共通している．特に，内部実装方式では user の増加が両環境で確認され，外部実装方式では CPU 使用率の増加が内部実装方式より大きいことが確認された．結果として，実装方式による性能低下の傾向は両環境で共通しているが，CPU 負荷の現れ方には環境による違いが見られた．

5.2　外部実装方式における柔軟性

外部実装方式は，DNS サーバ本体を変更することなく DNS 通信に対する機能を追加できるという特徴を有する．本方式では，DNS サーバ本体の実装に依存せず，DNS サーバとクライアント間の通信を中継する装置上で TC フラグ付与処理を実行できる．そのため，DNS サーバのソースコードを変更することなく DNS 通信に対する処理を柔軟に追加することができる．

一方，内部実装方式では TC フラグ付与処理を DNS サーバの応答生成処理に組み込むため，TC フラグの付与条件を変更する場合や新たな処理を追加する場合には，DNS サーバ本体の実装を変更する必要がある．そのため，使用する DNS サーバの実装やバージョンへの依存が生じる．

以上より，外部実装方式は内部実装方式と比較してスループットの低下が大きいという性能上の課題を有する一方，DNS サーバ本体を変更することなく機能を追加できるという特徴を持つ．したがって，外部実装方式では，性能と DNS サーバからの独立性の間にトレードオフが存在すると考えられる．

6.　本手法の限界及び今後の課題

6.1　本手法の限界

本研究では，TC フラグを付与することで TCP Fallback を誘導できることを前提として，その実装方式が DNS 性能に与える影響を評価した．一方で，実運用環境への適用を考えた場合，以下のような問題点が存在する．

- 性能上の限界

  外部実装方式では DNS サーバとクライアント間に TC フラグ付与装置を介在させるため，サーバ内の CPU 使用率が通常時と比べ増加するだけでなくスループットも大幅に低下する．内部実装方式の場合でも，外部実装方式と比較すると性能低下は小さいものの，通常の BIND と比較してスループットが仮想環境では 49.75%，実機環境では 36.04%低下しており，性能への影響を無視することはできない．したがって，内部実装方式においても，TC フラグ付与処理による計算負荷の増加が DNS サーバの処理性能に影響を与えるという問題がある．
- 可用性における課題

  外部実装方式では，DNS サーバとクライアント間に TC フラグ付与装置を配置し，DNS 通信を一度装置で受信した上で TC フラグの付与および DNS サーバへの転送を行う．そのため，通常の BIND のように DNS サーバのみで処理を行う構成と比較して DNS 通信を処理する装置が追加されることになる．この構成では，DNS サーバだけでなく TC フラグ付与装置にも処理負荷が発生するため，外部実装方式では通常の構成と比較して大量の通信による負荷の影響を受けやすくなる．

  特に，DoS（Denial of Service）攻撃などによって大量の DNS クエリが送信された場合，TC フラグ付与装置におけるパケットの受信，解析および転送処理が増加する．本研究では外部実装方式において大幅なスループット低下が確認されていることから，TC フラグ付与装置が DNS 通信のボトルネックとなり，攻撃による負荷が DNS サーバへ到達する前に装置の処理能力を超える可能性がある．その結果，正規の DNS クエリについても処理遅延やパケット損失が発生し，DNS サービス全体の可用性が低下する可能性がある．

6.2　今後の課題

本研究では，先行研究で示された TCP による DNS キャッシュポイズニング攻撃の抑制手法を実際の DNS 環境へ適用することを想定し，TC フラグを利用した TCP Fallback 誘導方式について，実装方式による性能への影響を評価した．一方で，実運用環境への適用を考えた場合，以下の課題が残されている．

- TC フラグ付与装置と DNS サーバの分離

  本研究では，TC フラグ付与装置とキャッシュ DNS サーバを同一マシン上で動作させている．そのため，外部実装方式では，TC フラグ付与処理と DNS サーバ

© 2026 Information Processing Society of Japan

の処理が同一マシン上で実行されることで，CPU 資源を共有している．実運用環境では，これらを異なるマシンに分離する構成も考えられるため，今後は TC フラグ付与装置とキャッシュ DNS サーバを別々のマシンに配置し，スループットおよび CPU 使用率への影響を評価する必要がある．

- TC フラグ付与条件の柔軟化

  本研究では，内部実装方式および外部実装方式のいずれも TC フラグを一律に付与する実装としている．そのため，今後は DNS サーバの状態や通信状況などに応じて任意のタイミングで TC フラグを付与できるよう，付与条件を柔軟に設定可能な仕組みを実装する必要がある．

- 外部実装方式の軽量化

  外部実装方式では，DNS パケットの受信，解析および転送処理によるオーバーヘッドが大きく，内部実装方式と比較して大幅な性能低下が確認された．そのため，今後はパケット処理の効率化や実装の軽量化を行い，外部実装方式における性能向上を図る必要がある．

- 内部実装方式における柔軟性の向上

  内部実装方式では，TC フラグ付与処理を BIND のソースコードに直接実装しているため TC フラグの付与条件を変更する場合にはソースコードの改修が必要となる．そのため，今後は DNS サーバの設定ファイルに TC フラグ付与の条件を記述できるようにし，運用環境に応じて柔軟に制御できる仕組みを実装する必要がある．

- ロードバランシングによる負荷分散

  内部実装方式および外部実装方式のいずれにおいても，TC フラグ付与による性能低下が確認された．そのため，実運用環境では単一の DNS サーバや TC フラグ付与装置に負荷が集中しないよう，複数の装置を配置してロードバランシングを行う必要がある．今後は，ロードバランシングを導入した構成における性能および可用性について評価する必要がある．

7. おわりに

本研究では，TCP Fallback を単なる通信方式の切り替えとしてではなく，DNS 通信における状態情報の伝達手段として利用することを目的として DNS 応答に TC フラグを付与する方式を提案した．また，TC フラグ付与機能を DNS サーバ内部に実装する方式と DNS サーバ外部に配置した装置に実装する方式について，仮想環境および実機環境を用いて性能評価を行った．

評価の結果，いずれの実装方式においても TC フラグ付与による性能低下が確認された．特に外部実装方式では，DNS パケットの受信，解析および転送処理によるオーバーヘッドの影響から，内部実装方式と比較して大幅なスループット低下が確認された．一方で，外部実装方式は DNS サーバ本体とは独立して TC フラグ付与処理を実行できるため，DNS サーバの状態や通信状況に応じた機能拡張を行いやすく，内部実装方式と比較して高い柔軟性を持つことを示した．

以上より，TC フラグを利用した DNS 通信の状態制御には，実装方式によって性能と柔軟性の間にトレードオフが存在することが明らかになった．今後は，DNS サーバやネットワークの状態に応じて TC フラグ付与を動的に制御する方式について検討し，通常時の性能低下を抑制するとともに TC フラグを状態情報として利用する通信制御への応用を目指す．

参考文献

[1] Afek,Y.,Berger,H.and Bremler-Barr,A.:POPS: From History to Mitigation of DNS Cache Poisoning Attacks, Proc. 34th USENIX Security Symposium (USENIX Security 2025),2025.

[2] Cricket Liu, Paul Albitz, ”DNS&BIND 5th Edition”, O’REILLY, 2008.

[3] JPRS, ”Kaminsky Attack の全て”, Japan Network Information Center, 2008

[4] 一般社団法人 日本ネットワークインフォメーションセンター, ”インターネット 10 分講座:DNSSEC”, https://www.nic.ad.jp/ja/newsletter/No43/0800.html, 2026 年 8 月 12 日最終閲覧.

[5] 阿久津賢宏, 山井成良, 金勇, ”複数 DNS キャッシュサーバへの並行名前解決による DNS キャッシュポイズニング対策手法”, マルチメディア, 分散協調とモバイルシンポジウム論文集 p625-632, 情報処理学会, 2020.

[6] Amir Herzberg, Haya Shulman. “Towards Adoption of DNSSEC : Availability and Security Challenges”. IACR Cryptology e Print Archive 2013.

[7] A. Herzberg and H. Shulman. ”Fragmentation Considered Poisonous, or: One-domain-to-rule-them-all.org,” 2013 IEEE Conference on Communications and Network Security (CNS), National Harbor, MD, 2013, pp. 224-232, DOI: 10.1109/CNS.2013.6682711.

[8] 太田 健也, 鈴木 恒彦. “DNS 第一フラグメント便乗攻撃の追検証と対策の検討”. 第 81 回全国大会講演論文集 2019(1)443-444.

[9] ”DNSSEC World Map”,https://stats.labs.apnic.net/dnssec, 2026 年 8 月 12 日最終閲覧.

[10] 内閣官房国家サイバー統括室, ”政府機関等における耐量子計算機暗号（PQC）利用に関する関係府省庁連絡会議 第 2 回議事要旨”, 令和 7 年 11 月 19 日, https://www.cas.go.jp/jp/seisaku/pqc/dai2/gijiyoshi.pdf, 2026 年 8 月 3 日最終閲覧.

[11] Japan Registry Services Co., Ltd., ”TC ビット”, https://jprs.jp/glossary/index.php?ID=0204, 2026 年 8 月 3 日最終閲覧.

[12] Japan Registry Services Co., Ltd., ”「512 の壁」を越える ～EDNS0 の概要と運用上の注意 ～”, https://jprs.jp/related-info/guide/topics-column/no8.html, 2026 年 8 月 3 日最終閲覧.

[13] ”DNS library in Go”, https://github.com/miekg/dns, 2026 年 8 月 13 日最終閲覧.

[14] ”dnsperf”, https://codeberg.org/DNS-OARC/dnsperf, 2026 年月 13 日最終閲覧.

© 2026 Information Processing Society of Japan